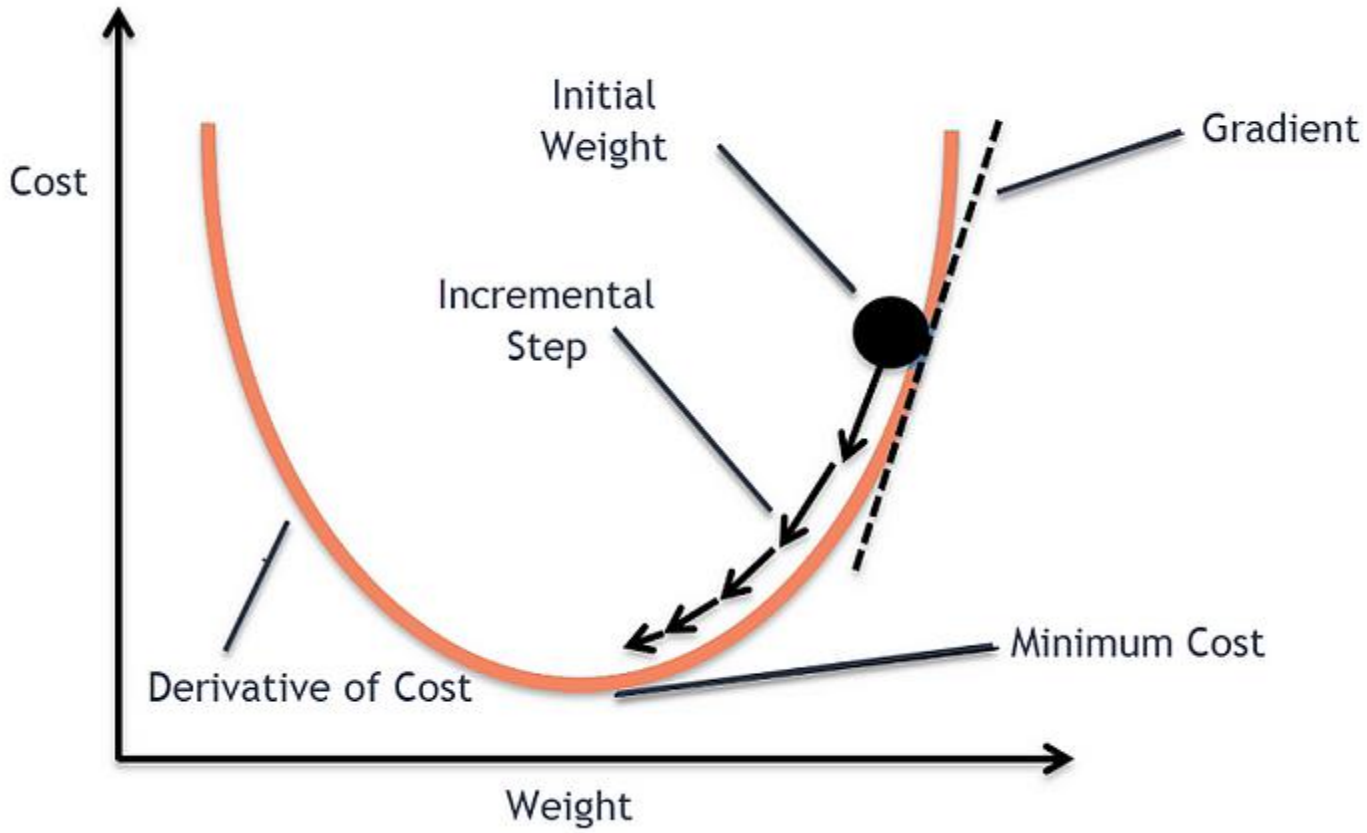
**Gradient Descent**

The goal of the gradient descent is to minimize a given function which, in our case, is the loss function of the neural network. To achieve this goal, it performs two steps iteratively.

1. Compute the slope (gradient) that is the first-order derivative of the function at the current point
2. Move-in the opposite direction of the slope increase from the current point by the computed amount



So, the idea is to pass the training set through the hidden layers of the neural network and then update the parameters of the layers by computing the gradients using the training samples from the training dataset.

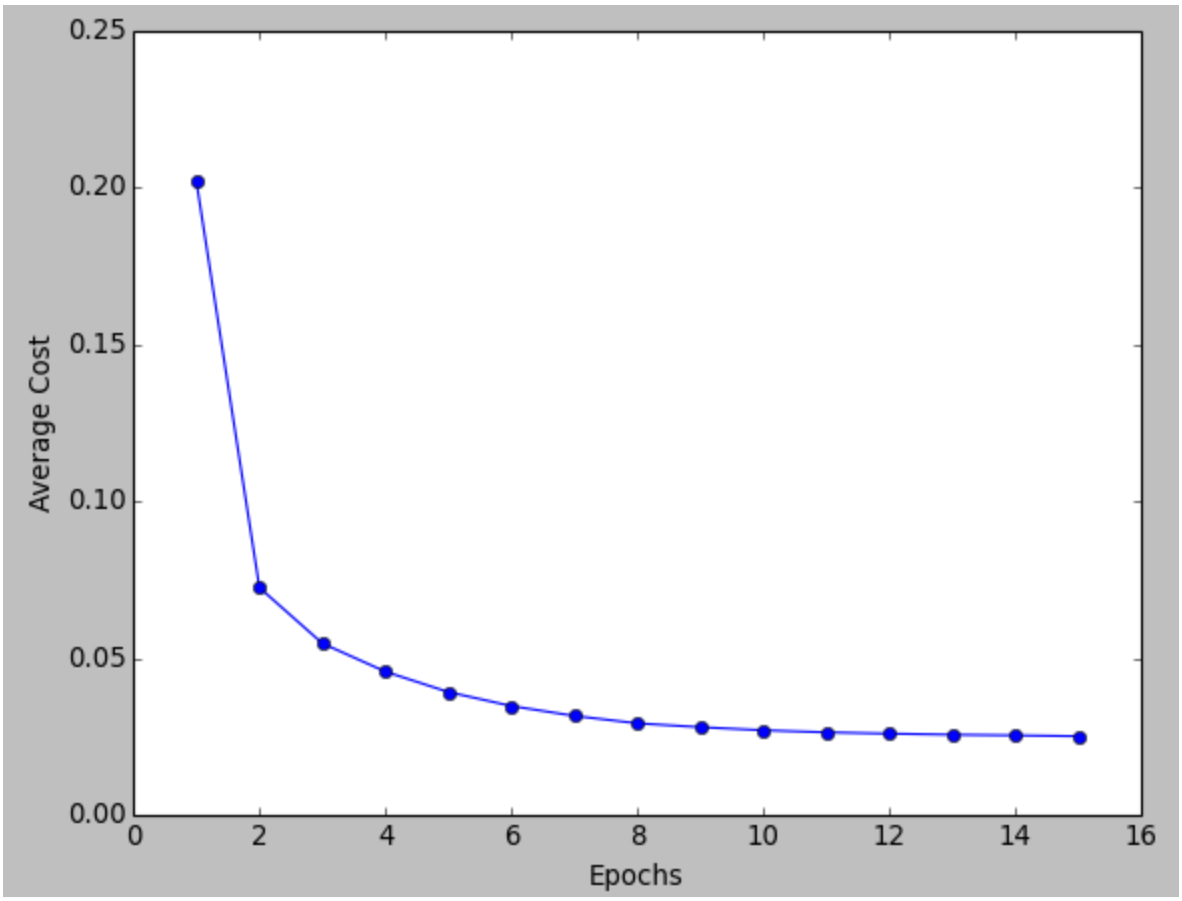
Think of it like this. Suppose a man is at top of the valley and he wants to get to the bottom of the valley. So he goes down the slope. He decides his next position based on his current position and stops when he gets to the bottom of the valley which was his goal.

There are different ways in which that man (weights) can go down the slope. Let's look into them one by one.

**Batch Gradient Descent**

In Batch Gradient Descent, all the training data is taken into consideration to take a single step. We take the average of the gradients of all the training examples and then use that mean gradient to update our parameters. So that's just one step of gradient descent in one epoch.

Batch Gradient Descent is great for convex or relatively smooth error manifolds. In this case, we move somewhat directly towards an optimum solution.




Cost vs Epochs

The graph of cost vs epochs is also quite smooth because we are averaging over all the gradients of training data for a single step. The cost keeps on decreasing over the epochs.

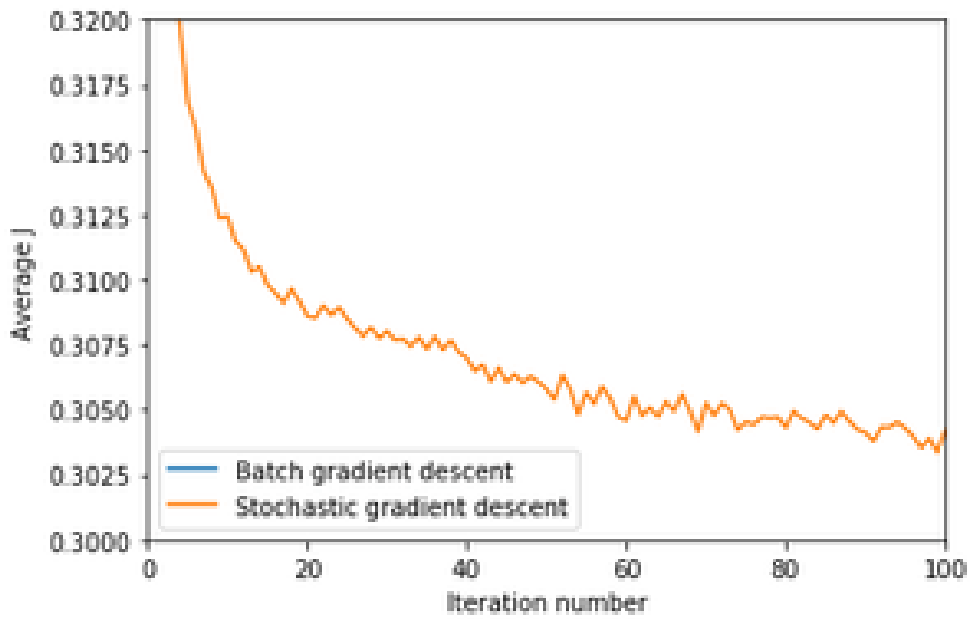
**Stochastic Gradient Descent**

In Batch Gradient Descent we were considering all the examples for every step of Gradient Descent. But what if our dataset is very huge. Deep learning models crave for data. The more the data the more chances of a model to be good. Suppose our dataset has 5 million examples, then just to take one step the model will have to calculate the gradients of all the 5 million examples. This does not seem an efficient

way. To tackle this problem we have Stochastic Gradient Descent. In Stochastic Gradient Descent (SGD), we consider just one example at a time to take a single step. We do the following steps in **one epoch** for SGD:

1. Take an example
2. Feed it to Neural Network
3. Calculate it's gradient
4. Use the gradient we calculated in step 3 to update the weights
5. Repeat steps 1–4 for all the examples in training dataset

Since we are considering just one example at a time the cost will fluctuate over the training examples and it will **not** necessarily decrease. But in the long run, you will see the cost decreasing with fluctuations.




Also because the cost is so fluctuating, it will never reach the minima but it will keep dancing around it.

SGD can be used for larger datasets. It converges faster when the dataset is large as it causes updates to the parameters more frequently.

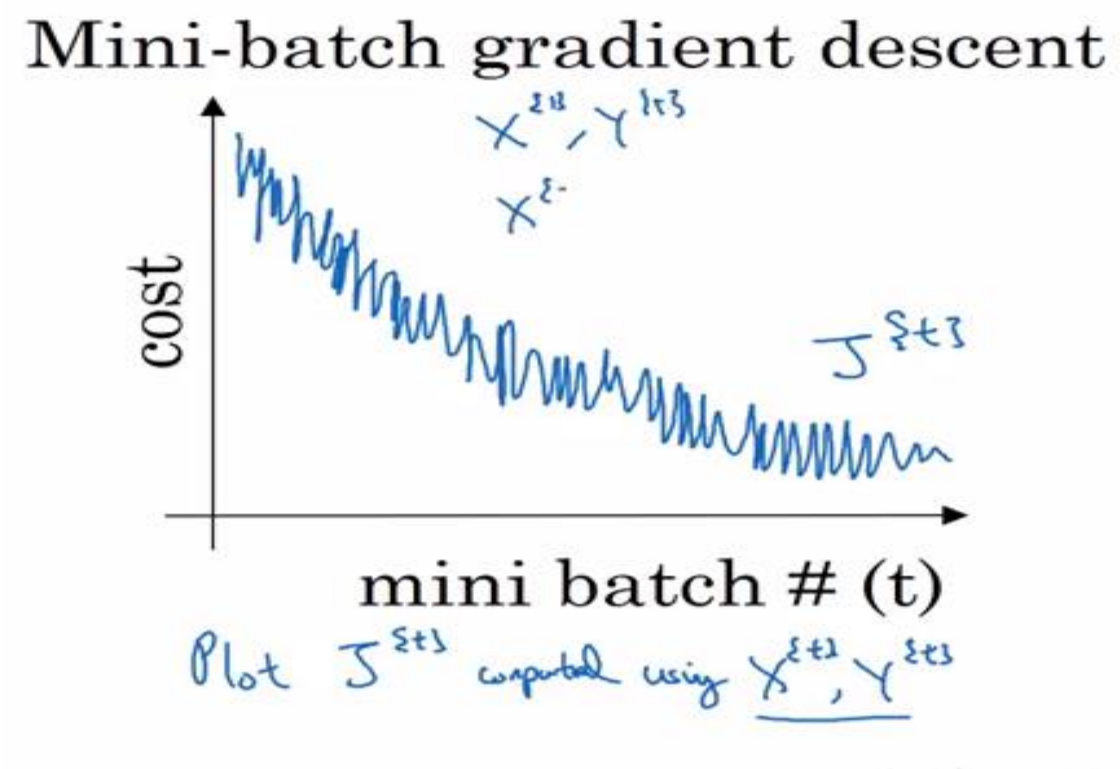
**Mini Batch Gradient Descent**

We have seen the Batch Gradient Descent. We have also seen the Stochastic Gradient Descent. Batch Gradient Descent can be used for smoother curves. SGD can be used when the dataset is large. Batch Gradient Descent converges directly to minima. SGD converges faster for larger datasets. But, since in SGD we use only one example at a time, we cannot implement the vectorized implementation on it. This can slow down the computations. To tackle this problem, a mixture of Batch Gradient Descent and SGD is used.

Neither we use all the dataset all at once nor we use the single example at a time. We use a batch of a fixed number of training examples which is less than the actual dataset and call it a mini-batch. Doing this helps us achieve the advantages of both the former variants we saw. So, after creating the mini-batches of fixed size, we do the following steps in **one epoch:**

1. Pick a mini-batch
2. Feed it to Neural Network
3. Calculate the mean gradient of the mini-batch
4. Use the mean gradient we calculated in step 3 to update the weights
5. Repeat steps 1–4 for the mini-batches we created

Just like SGD, the average cost over the epochs in mini-batch gradient descent fluctuates because we are averaging a small number of examples at a time.




## Ridge, LASSO, and ElasticNet Regression

**The continuing adventures of regularization and the eternal quest to prevent model overfitting!**

How do we simplify our models? By removing as many features as possible. Why do I want to do this? I want to have a simpler model. Why do I want a simpler model? Well, because a simpler model generalizes better. What does generalization mean? **It means that you can actually use it in the real world**. The model does not just explain the training data but it also has a better chance of explaining data that we have not seen before.

**Regularization**

Regularization seeks to control variance by adding a tuning parameter, lambda, or alpha:

**LASSO (L1 regularization)**

- regularization term penalizes absolute value of the coefficients
- sets irrelevant values to 0
- might remove too many features in your model

**Ridge regression (**L2 regularization**)**

- penalizes the size (square of the magnitude) of the regression coefficients
- enforces the *B* (slope/partial slope) coefficients to be lower, but not 0
- does not remove irrelevant features, but minimizes their impact

Now do not worry about that lambda. For right now assume lambda is one. For ridge regression, we penalize the size of the regression coefficient. So a larger regression coefficient is a larger penalty.

$$Y = a + b * X1 + c * X2$$

b and c are the regression coefficients

So in this ridge regression, I would take the square of **b** and add it to the square of **c**. And that would be my penalty. Now it is slightly more complex than that because

sometimes I want to do make the penalty stronger or less strong, I now introduce this lambda, this tuning parameter. So my penalty now becomes:

$$Ridge\ Regularization\ Penalty : \lambda * b^2 + \lambda * c^2; ||w||2$$

Otherwise known as the L2 or Euclidian norm

So this is what the Ridge regression does. By adding this penalty to my cost function, during training, it's going to keep these b's and these c's small. And by doing so this is going to simplify the model; that means any given column is not going to have that much power.

LASSO does pretty much the same thing. Except for the regularization penalty is without the square:

$$LASSO\ Regularization\ Penalty : \lambda * |b| + \lambda * |c|; ||w||$$

Otherwise known as the L1 or Manhattan/taxicab norm

Please take note that because you could have negative values here, so you need to add an absolute value on the coefficients, on the partial slopes.

The coefficients b, and c, in the above example, are adjusted by the algorithm you are using. We add another quirk to it by introducing a lambda penalty that is not 1 — it has always been there behind the scenes but we kept it at 1 for simplicity. When it comes time to evaluate the goodness of your model, through the cost function, we are going to multiply a penalty we called lambda to your coefficients (or to the square of your coefficients), and that will be an additional penalty.

We still have our general cost function. This is just an addition to that cost function. Our general cost function is always the same and it is the one where we have y — predicted y and take the square of that and sum all those up. So that is our cost function, the baseline. Now, the additional penalty in order to regularize is either this Ridge regression, which uses the so-called L2 norm, or the LASSO (least absolute shrinkage and selection operator) regression, which uses the so-called L1 norm.

For both types of regression, a larger coefficient penalizes the model. With Ridge, the penalty is disproportionately larger because of the square, but in general, a larger coefficient reduces model performance, a larger coefficient reduces the goodness of fit.

N.B: The ***L*** notation is a reference to Minkowski norms and ***Lp*** spaces. These just generalize the notion of taxicab and Euclidean distances to *p>0p>0* in the following expression:

$$||x||_p = \left(|x_1|^p + |x_2|^p + \ldots + |x_n|^p\right)^{\frac{1}{p}}$$

For those of you who remember back to Linear Algebra

**Ridge regression**

Reiterating again, Ridge regression is a way to limit the number of independent variables (columns/features/attributes) in the regression. The regular least-squares criterion minimizes the least-squares of the error plus a regularization term that is a product of a constant and the sum of squared coefficients:

$$min \sum (y - y_i)^2 + \alpha * \sum \beta^2$$

Ridge regression cost function, expressed another way | alpha is sometimes used instead of lambda in certain literature

Essentially this is preventing the partial slope terms from getting too large.

Here I am calling this penalty coefficient alpha as opposed to lambda. And this is a little bit confusing since alpha has a few meanings that are very closely related in this jargon. So just want to warn you, in general, you might have some problems, lambda is a safe bet. Alpha has at least two very closely, but still different meanings.

Here, we have this beta squared. What is beta? Beta is a vector of your coefficients. So beta is a vector that is basically in my would have been b and c in my previous example. Remember, I had y = a +b*X1 + c*X2, b and c were two coefficients. Beta is a vector, and here composed of two numbers. In my example, those two numbers are b and c; a is the intercept, sometimes confusingly referred to as b.

So beta squared is exactly the same thing as the equation above. What is a partial slope? Another word for partial slope is a coefficient. Why do we call it a partial slope? Well, if you remember, we have a slope when we have a linear regression, y = a + b*X, b is the slope. Well, if you have multiple x's, so y = a + b*X1 + c*X2 + d*X3 and so on and so on. Then, in my example, that I just mentioned b, c, and d are all partial slopes, that is because they are multiplied by the columns. And the way the input value from the column changes from those variables change will affect the

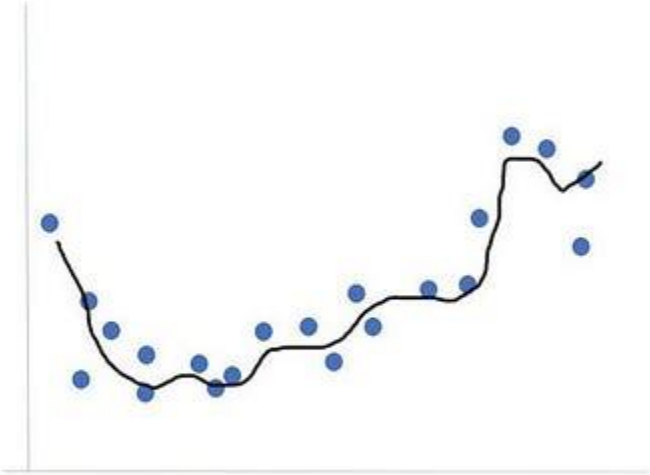
output. And it will affect it more if that coefficient or that partial slope is greater. When I say greater, I mean in magnitude.

So the Ridge objective **is the residual sum of squares (RSS) +lambda * (sum of squares of the coefficients)**.

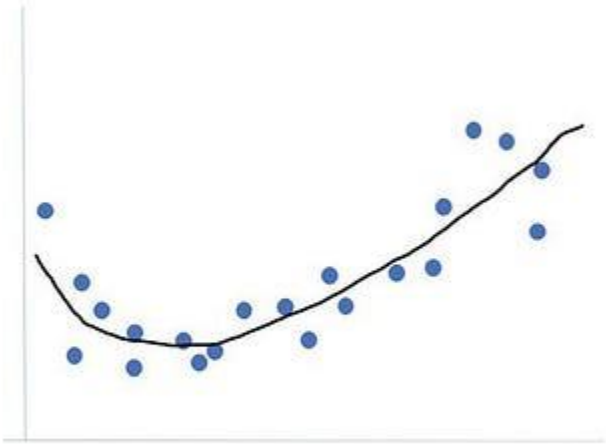
- use lambda to balance minimizing RSS vs. minimizing the coefficients
- when lambda = 0, results are the same as regular linear regression: you have removed the Ridge regression penalty
- as lambda increases to infinity, coefficients become closer to 0
- any non-zero lambda value will shrink the coefficients to be smaller than with non-regularized regression
- as lambda (alpha) increases, model complexity decreases

Lambda is just a scalar that should be learned from your data using cross-validation (tuning). So lambda is always positive and is between zero and infinity. And just to give you a basic idea of its range, it is rarely much larger than one, maybe going to two or three, and often is a small fraction of one. Why is that? Well, that is just the way things work out. There exist counterexamples to what I just said, but as a general heuristic, what you generally find is that the lambda is between zero and, and a small value, let us say roughly two.

If the model complexity decreases as lambda increases, how can you see this?



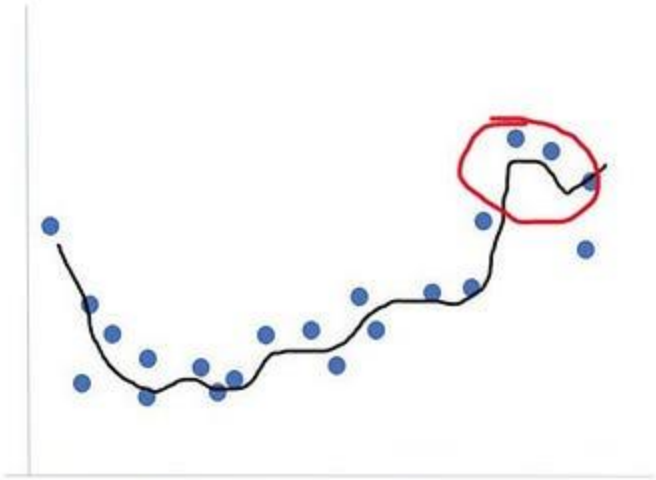
Plot for lambda: 1e-15



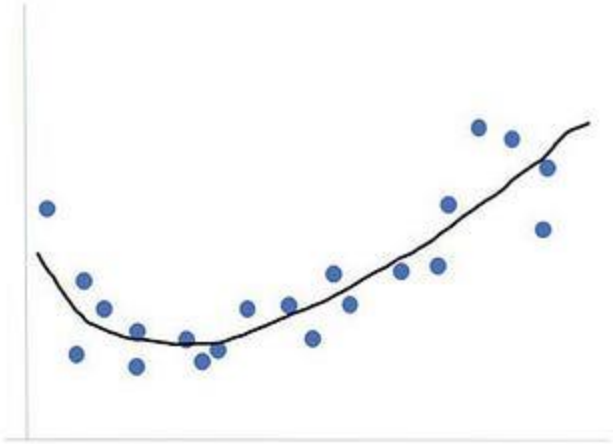
Plot for lambda: 0.01

**Our regression lines fit the blue dots well**

In the example fits above I approximate some form of regression, in this case, probably not a linear regression. And that regression worked pretty well. And then we added the lambda parameter. And hopefully, things got better. If I were to eyeball this here, I would probably want the one on the right. My rationale is, I see these blue dots, they are flying all over the place, they are up and down above and beneath that black line. I expect there to be overall, more variation in where those blue dots can land, a lot of variation. And so I think that trying to find the model that tries to minimize the residuals of these blue dots will try to minimize it too much, what will happen is it will become too complex of a model, it will overfit. And then when new data comes by, data that the model has never seen, that new data is not going to be there where this model is fitting.



Plot for lambda: 1e-15



Plot for lambda: 0.01

**The red circle is likely overfitting the residuals**

So for instance, this little quirk circled in red, looks pretty good. But my guess is that is just because one or two data points had these high y values relatively - high y values. Methinks that was just a quirk. Probably this should just go straight over, as it does on the right-most image. Because if I look at how much overall these blue points are going up and down, I cannot really justify creating a model that is trying to go down to catch to minimize the residual of this one bluepoint and then going right back up. As such, the small lambda value is likely overfitting and I believe that the 1/100 lambda value on the right actually fits the data much better and will generalize better.

**LASSO**

LASSO regression does pretty much the same thing; it is another way to limit the number of independent variables in the regression. The only difference is, it uses the absolute value. Now the thing is that because of the math, what it does is not just reduce the coefficients, LASSO actually removes the coefficients. Many people like LASSO for this reason: **the coefficients can actually go to zero, whereas in the Ridge regression, they do not go to zero, they become small, but they never vanish**.

Now, you can always help the last way, if you see that coefficient has gotten really small, you can just say, "Okay, I'm going to just toss this column, because it has such a small coefficient." What is nice about LASSO is that it will actually do that for you in an intelligent way.

$$min \sum (y - y_j)^2 \; such \; that \; \sum |\beta_i| < \lambda$$

y is the predicted for j points, there are i terms with beta coefficients — lambda limits the betas

Beta a vector of your coefficients. Instead of squaring them, you just take the absolute value.

## Linear algebra

Next is a quick review of linear algebra. Why do we have a quick review of linear algebra? Well, because machine learning is a lot of linear algebra and the more you understand about the machine learning algorithms, the often the better you can apply

them. **When studying data science, you are not studying machine learning, you are studying how to use the machine learning algorithms**.

Algebraic properties of matrices:

- add/subtract matrices: must be of the same dimensions (m * n)
- for multiplication of matrices: inner dimensions must match
- remember that matrix multiplication is not commutative: A x B != B x A
- a matrix multiplied by its inverse yields the identity matrix
- matrix division is not defined

**Linear regression with matrices**

Consider the linear regression formula:

$$y_i = m * x_i + b + \epsilon, \epsilon \approx N(0, \sigma^2)$$

written as a matrix:

$$\begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} mx_1 + b \\ \vdots \\ mx_n + b \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

which can be rewritten as:

$$\begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \begin{bmatrix} b \\ m \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

For linear regression with matrices, if you have, say 10 cases, that means 10 points and you will have 10 columns, then there will be an exact, best solution. There will be a simple solution to that. However, not in every case, there are cases where you have colinearity, which means when one column is basically the same as another column or the same as a combination of some other columns. When this occurs you generally have infinitely many solutions. And we do not like this because having infinitely many solutions, means you do not have any useful solutions. If then there are situations where there is simply no solution can occur. But the general case or the case that you hope for is that you have exactly one good solution.

Most of the time, when we do machine learning, or data science is we have many more cases than we have variables; we have many more data points than we have columns; we have many more rows than we have columns — I am just repeating the same thing over and over with slightly different words. This is going to cause us an issue, for one thing, that means our matrix of x is not a square matrix.

So when we want to invert it, and I will show you that we will want to do that, it is going to cause us some problems, along with colinearity and stuff like that.

$$\begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \begin{bmatrix} b \\ m \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

Vector of Responses

Design Matrix

Vector of Parameters

Vector of Residuals (error terms)